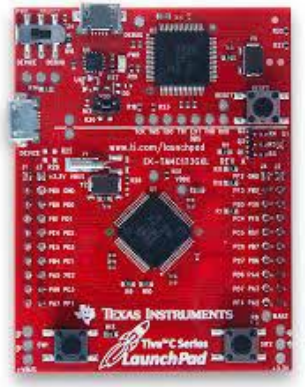
SDU

# Recap of deadlocks / reentrancy
# Assembler in C
# FPU



# Today’s agenda

- Assignment08 presentation
- Quizz
- Recap deadlocks / reentrancy
- Assembly in C
- Short mention of FPU
- Course feedback/evaluation
- Intro to Final Assignment
- Work on Final Assignment

SDU

# Course outline

- **The plan is indicative, subject to change!**

| Date | Lecture | Subject(s) | Lab | Assignment due |
|---|---|---|---|---|
| Feb 2 | 1 | Introduction; ARM Cortex-M4 | Lab1: Setting up the development env. | |
| Feb 9 | 2 | EMP coding standard; Bit manipulation | Lab2: Bit manipulation | |
| Feb 16 | 3 | State machines; The compiler chain; EMP-Board | Lab3: State machines | PF1 – Assignment 1 |
| Feb 23 | 4 | The task model; The pre-processor | Lab4: A clock radio task model | PF1 – Assignment 2 |
| Mar 2 | 5 | Queues and semaphores; Debugging | Lab5: Debug with a serial connection | PF1 – Assignment 3 |
| Mar 9 | 6 | Run to complete scheduler | Lab6: RTCS, Run to compile scheduler | PF1 – Assignment 4 |
| Mar 16 | 7 | More debugging; C: printf() | Lab7: RTCS, Debugger | PF1 – Assignment 5 |
| March 23 | 8 | FreeRTOS | Lab8: FreeRTOS | PF1 – Assignment 6 |
| March 30<br>April 6 | | Easter holiday | | |
| Apr 13 | 9 | Re-entrance, more queues | Lab9: FreeRTOS (continued) | PF1 – Assignment 7 |
| Apr 20 | 10 | Assembler in C, | Work on the final assignment | PF1 – Assignment 8 |
| Apr 27 | 11 | Work on the final assignment, consultations | | |
| May 4 | 12 | Work on the final assignment, consultations | | |
| May 11 | 13 | Poster session | | PF2- Final Assignment |



SDU

# Quizzzz time!

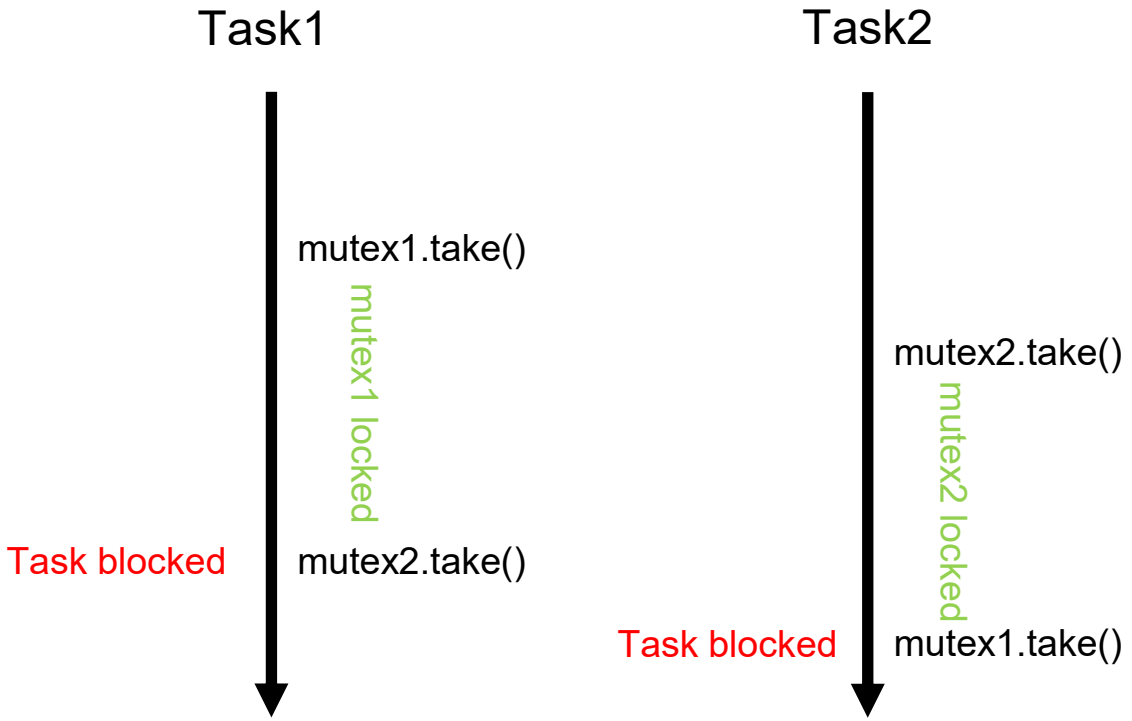
- Go to Slido.com
- Enter code: #025622

# Recap - deadlocks



# Recap - deadlock in other words

- A process cannot proceed because it needs to obtain a resource held by another process but it itself is holding a resource that the other process needs



# Recap Coffman's 4 conditions of deadlock

- **Mutual exclusion** – a resource can be held by at most one process
- **Hold and wait -** processes that already hold a resource can wait for another resource
- **Non-preemption** – a resource, once granted, cannot be taken away
- **Circular wait** – two or more processes are waiting for resources held by one of the other processes



SDU

# Recap - Reentrance

- A function is reentrant or pure if, while it is being executed, can be called again by itself or by another function
- …and it works every time
- i.e. more than one instance of a function can run at the same time

SDU

# Recap - Recursion

- a function is recursive, if it calls itself
- a recursive function must necessarily be reentrant
- A reentrant function must not necessarily be recursive

```
double power( double x, int exp )
{
     if( exp <= 0 )
          return(1);
     return( x*power(x, exp-1));
}
```

SDU

# Assember in C



SDU

# What is assembler?

- Low-level (close to hardware) programming language
- Assembly code - very close to machine code instructions
- Direct mapping from assembler instructions to processor instructions
- Additionally, it contains macros, directives, comments, symbolic names of memory locations, etc.
- Source code needs to be ‘assembled’ into binary machine code
- Naming:
  - Assembly Language – the human-readable programming language that contains mnemonics of machine code (mov, ldr, etc.)
  - Assembler – computer program that translates assembly source code into machine language object code



SDU

# Why use assembler?

- Optimization
  - Speed
  - Size
- Strict timing
  - For time critical tasks
  - Real-time
  - No ambiguity about how the code will be complied



SDU

# ARM Cortex M4 Instruction Set

- Instruction set summary:
  - ARM Cortex-M4 Technical Reference Manual
    - Search for ‘instruction set summary’

- Detailed description of instructions:
  - https://developer.arm.com/docs/dui0553/a/the-cortex-m4-instruction-set



SDU

# ARM Cortex M4 Instruction Set

| Operation | Description | Assembler | Cycles | Notes |
|---|---|---|---|---|
| Move | Register | `MOV Rd, <op2>` | 1 | |
| | 16-bit immediate | `MOVW Rd, #<imm>` | 1 | |
| | Immediate into top | `MOVT Rd, #<imm>` | 1 | |
| | To PC | `MOV PC, Rm` | 1 + P | |
| Add | Add | `ADD Rd, Rn, <op2>` | 1 | |
| | Add to PC | `ADD PC, PC, Rm` | 1 + P | |
| | Add with carry | `ADC Rd, Rn, <op2>` | 1 | |
| | Form address | `ADR Rd, <label>` | 1 | |
| Subtract | Subtract | `SUB Rd, Rn, <op2>` | 1 | |
| | Subtract with borrow | `SBC Rd, Rn, <op2>` | 1 | |
| | Reverse | `RSB Rd, Rn, <op2>` | 1 | |
| Multiply | Multiply | `MUL Rd, Rn, Rm` | 1 | |
| | Multiply accumulate | `MLA Rd, Rn, Rm` | 1 | |
| | Multiply subtract | `MLS Rd, Rn, Rm` | 1 | |
| | Long signed | `SMULL RdLo, RdHi, Rn, Rm` | 1 | |
| | Long unsigned | `UMULL RdLo, RdHi, Rn, Rm` | 1 | |
| | Long signed accumulate | `SMLAL RdLo, RdHi, Rn, Rm` | 1 | |
| | Long unsigned accumulate | `UMLAL RdLo, RdHi, Rn, Rm` | 1 | |
| Divide | Signed | `SDIV Rd, Rn, Rm` | 2 to 12 | Division operations terminate when the divide calculation completes, with the number of cycles required dependent on the values of the input operands. Division operations are interruptible, meaning that an operation can be abandoned when an interrupt occurs, with worst case latency of one cycle, and restarted when the interrupt completes. |
| | Unsigned | `UDIV Rd, Rn, Rm` | 2 to 12 | |
| Saturate | Signed | `SSAT Rd, #<imm>, <op2>` | 1 | |
| | Unsigned | `USAT Rd, #<imm>, <op2>` | 1 | |
| Compare | Compare | `CMP Rn, <op2>` | 1 | |
| | Negative | `CMN Rn, <op2>` | 1 | |
| Logical | AND | `AND Rd, Rn, <op2>` | 1 | |
| | Exclusive OR | `EOR Rd, Rn, <op2>` | 1 | |
| | OR | `ORR Rd, Rn, <op2>` | 1 | |
| | OR NOT | `ORN Rd, Rn, <op2>` | 1 | |
| | Bit clear | `BIC Rd, Rn, <op2>` | 1 | |
| | Move NOT | `MVN Rd, <op2>` | 1 | |
| | AND test | `TST Rn, <op2>` | 1 | |
| | Exclusive OR test | `TEQ Rn, <op1>` | | |
| Shift | Logical shift left | `LSL Rd, Rn, #<imm>` | 1 | |
| | Logical shift left | `LSL Rd, Rn, Rs` | 1 | |
| | Logical shift right | `LSR Rd, Rn, #<imm>` | 1 | |
| | Logical shift right | `LSR Rd, Rn, Rs` | 1 | |
| | Arithmetic shift right | `ASR Rd, Rn, #<imm>` | 1 | |
| | Arithmetic shift right | `ASR Rd, Rn, Rs` | 1 | |
| Rotate | Rotate right | `ROR Rd, Rn, #<imm>` | 1 | |
| | Rotate right | `ROR Rd, Rn, Rs` | 1 | |
| | With extension | `RRX Rd, Rn` | 1 | |
| Count | Leading zeroes | `CLZ Rd, Rn` | 1 | |
| Load | Word | `LDR Rd, [Rn, <op2>]` | 2 | Neighboring load and store single instructions can pipeline their address and some cases, such as 32- on odd halfword ight not pipeline |
| | | | | completes in a single is not taken. |
| Branch | Conditional | `B<cc> <label>` | | |
| | Unconditional | `B <label>` | | |
| | With link | `BL <label>` | | |
| | With exchange | `BX Rm` | | |
| | With link and exchange | `BLX Rm` | | |
| | Branch if zero | `CBZ Rn, <label>` | | |
| | Branch if non-zero | `CBNZ Rn, <label>` | | |
| | Byte table branch | `TBB [Rn, Rm]` | | |
| | Halfword table branch | `TBH [Rn, Rm, LSL#1]` | | |
| | Supervisor call | `SVC #<imm>` | | |



SDU

# Mixing C and Assembler

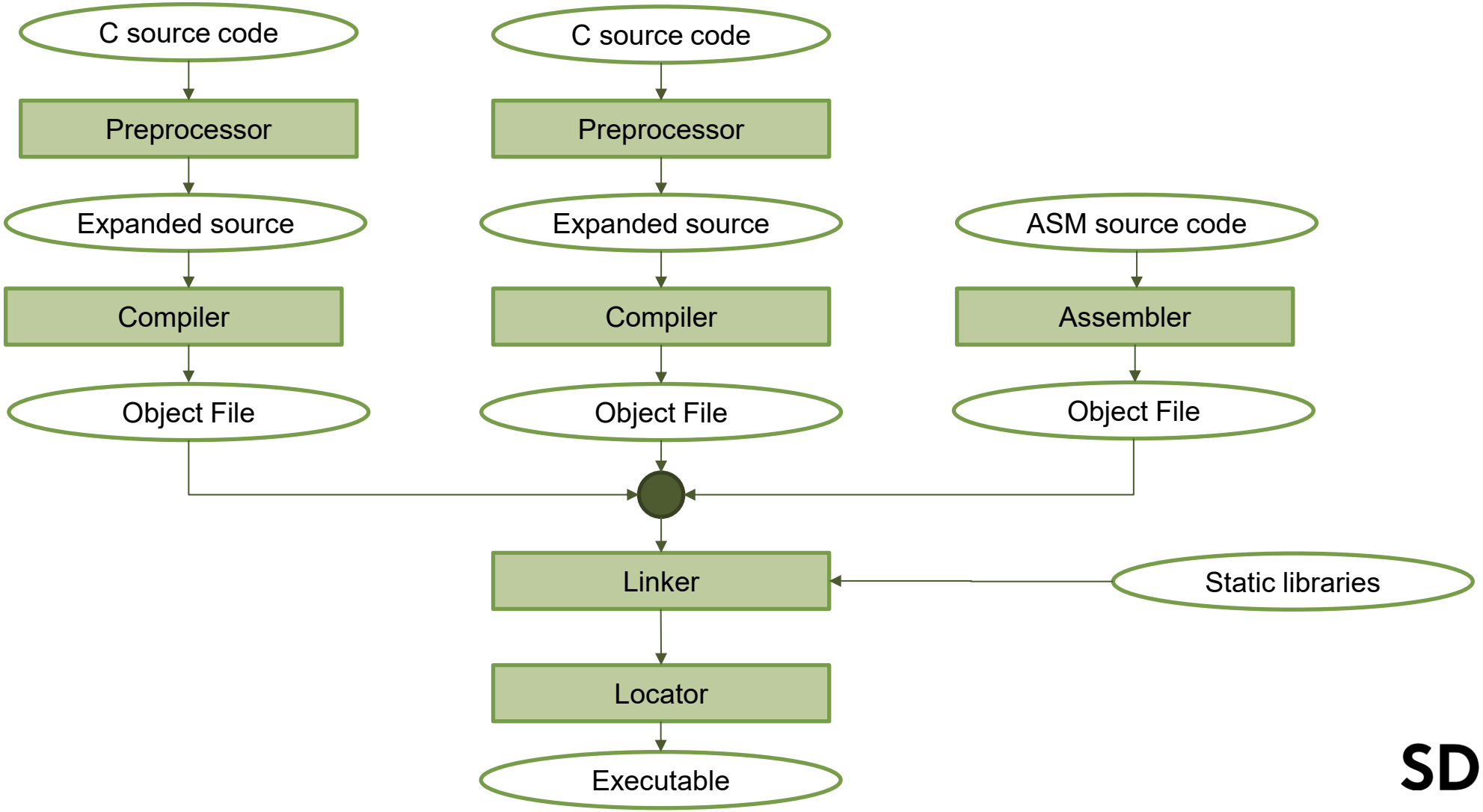
- Inline Assembler
  - Appears in .c code
  - Example: `asm("ldr r3, [r7, #6]");`

- Linked Assembler
  - Separate assembler source files which are linked at build-time
  - Example: `Label: ldr r3, [r7, #6] ; Comment`



SDU

# Linked Assembler – build process



# Calling convention: Procedure Call Standard

- where parameters and return values are placed (in registers; on the call stack; a mix of both)
- the order in which parameters are passed (or parts of a single parameter)
- how the task of setting up for and cleaning up after a function call is divided between the caller and the callee
- Procedure Call Standard for the Arm® Architecture used by the Application Binary Interface
- ABI – interface between two programming modules



SDU

# ASM Examples

- With global variables
- With parameters passed



SDU

# SetBits

```
unsigned char SetBits( unsigned char Byte, unsigned char Mask, unsigned char Target );
//-------------------------------------------------------
//Set the bits marked by Mask in Target to the bit values given by Byte.
//Return the new value of the port.
//-------------------------------------------------------
main()
{
        unsigned char Image;
        Image = SetBits( 0x29, 0x38, 0xF0 );
}
```



SDU

# SetBitsCGlobal

```
unsigned char GlobalByte;
unsigned char GlobalMask;
unsigned char GlobalTarget;
unsigned char GlobalTemp;
unsigned char GlobalTemp1;

void SetBitsCGlobal( )
{
    GlobalTemp = GlobalByte & GlobalMask;          // 00101000
    GlobalMask ^= 0xFF;                            // 11000111
    GlobalTemp1 = GlobalTarget & GlobalMask;       // 11000000
    GlobalTarget = GlobalTemp + GlobalTemp1;       // 11101000
}

main()
{
    unsigned char Image;
    GlobalByte = 0x29;        // 00101001
    GlobalMask = 0x38;        // 00111000
    GlobalTarget = 0xf0;      // 11110000
    SetBitsCGlobal();
    Image = GlobalTarget;     // 11101000
}
```



SDU

# SetBitsASMGlobal

```
void SetBitsASMGlobal( )
{
    // GlobalTemp = GlobalByte & GlobalMask
    asm( "ldr r3, =GlobalByte" );       // load address of GlobalByte to r3
    asm( "ldrb r2, [r3, #0]" );         // load value of GlobalByte to r2
    asm( "ldr r3, =GlobalMask" );       // load address of GlobalMask to r3
    asm( "ldrb r3, [r3, #0]" );         // load value of GlobalMask to r3
    asm( "and r3, r2, r3" );            // bitwise and on Byte and Mask
    asm( "ldr r2, =GlobalTemp" );       // load address of GlobalTemp to r2
    asm( "strb r3, [r2, #0]" );         // save r3 to GlobalTemp
    // GlobalMask ^= 0xFF
    asm( "ldr r3, =GlobalMask" );       // load address of GlobalMask to r3
     asm( "ldrb r3, [r3, #0]" );        // load value of GlobalMask to r3
     asm( "mvn r3, r3" );               // bitwise not and move r3 to r3
     asm( "ldr r2, =GlobalMask" );      // load address of GlobalMask to r2
     asm( "strb r3, [r2, #0]" );        // store value of r3 to GlobalMask
     //GlobalTemp1 = GlobalTarget & GlobalMask;
     asm( " ldr r3, =GlobalTarget" );   // load address of GlobalTarget to r3
     asm( "ldr r2, [r3, #0]" );         // load value of GlobalTarget to r2
     asm( "ldr r3, =GlobalMask" );      // load address of GlobalMask to r3
     asm( "ldrb r3, [r3, #0]" );        // load value of GlobalMask to r3
     asm( "and r3, r2, r3" );           // bitwise and between GlobalTarget and GlobalMask
     asm( "ldr r2, =GlobalTemp1" );     // load address of GlobalTemp1 to r2
     asm( "strb r3, [r2, #0]" );        // store value of and operation to GlobalTemp1
     // GlobalTarget = GlobalTemp + GlobalTemp1;
     asm( "ldr r1, =GlobalTarget" );    // load address of GlobalTarget to r1
     asm( "ldr r3, =GlobalTemp" );      // load address of GlobalTemp to r3
     asm( "ldrb r3, [r3, #0]" );        // load value of GlobalTemp to r3
     asm( "mov r2, r3" );               // move r3 to r2
     asm( "ldr r3, =GlobalTemp1" );     // load address of GlobalTemp1 to r3
     asm( "ldrb r3, [r3, #0]" );        // load value of GlobalTemp1 to r3
     asm( "add r3, r2, r3" );           // add GlobalTemp to GlobalTemp1
     asm( "str r3, [r1, #0]" );         // store result of addition to GlobalTarget
}
```



SDU

# Argument passing

- The base standard provides for passing arguments in core registers (r0-r3) and on the stack
- For subroutines that take a small number of parameters, only registers are used, greatly reducing the overhead of a call.
- The argument list for a subroutine call is formed by taking the user arguments in the order in which they are specified.



SDU

# SetBitsCParam

```
unsigned char SetBitsCParam(unsigned char byte, unsigned char mask, unsigned char target )
{
    unsigned char temp = byte & mask;
    mask ^= 0xFF;
    unsigned char temp1 = target & mask;
    target = temp + temp1;
    return target;
}
main()
{
    unsigned char Image;
    unsigned char Byte = 0x29;        // 00101001
    unsigned char Mask = 0x38;        // 00111000
    unsigned char Target = 0xf0;      // 11110000
    Target = SetBitsCParam(Byte, Mask, Target);
    Image = Target;                   //11101000
}
```



SDU

# SetBitsCParam

```
unsigned char SetBitsASMParam(unsigned char byte, unsigned char mask, unsigned char target )
{
    unsigned char temp;       // param1 byte -> r0, param2 mask -> r1
    unsigned char temp1;      // param3 target -> r2
    // temp = byte & mask
    asm( "mov r4, r0" );              // move param byte from r0 to r4
    asm( "mov r3, r1" );              // move param mask from r1 to r3
    asm( "and r3, r4, r3" );          // bitwise and between byte and mask to r3
    asm( "strb r3, [r7, #15]" );      // store result of and to temp, r7 points to stack
    // GlobalMask ^= 0xFF
    asm( "mvn r1, r1" );              // bitwise not to mask
    //temp1 = target & mask;
    asm( "and r3, r1, r2" );          // bitwise and between not-mask and target (r2)
    asm( "strb r3, [r7, #14]" );      // store value of and to temp1
    // target = temp + temp1;
    asm( "ldrb r3, [r7, #14]" );      // load temp1 to r3
    asm( "ldrb r4, [r7, #15]" );      // load temp to r4
    asm( "add r3, r3, r4" );          // add temp and temp1
    asm( "strb r3, [r7, #15]" );      // store result of addition to temp
    return temp;                      // return temp
}
main()
{
    unsigned char Image;
    unsigned char Byte = 0x29;        // 00101001
    unsigned char Mask = 0x38;        // 00111000
    unsigned char Target = 0xf0;      // 11110000
    Target = PortBitASMParam(Byte, Mask, Target);
    Image = Target;                   //11101000
}
```

| | |
|---|---|
| 15 | Local var1 (temp) |
| 14 | Local var2 (temp1) |
| 13 | |
| … | |
| 1 | |
| 0 | |

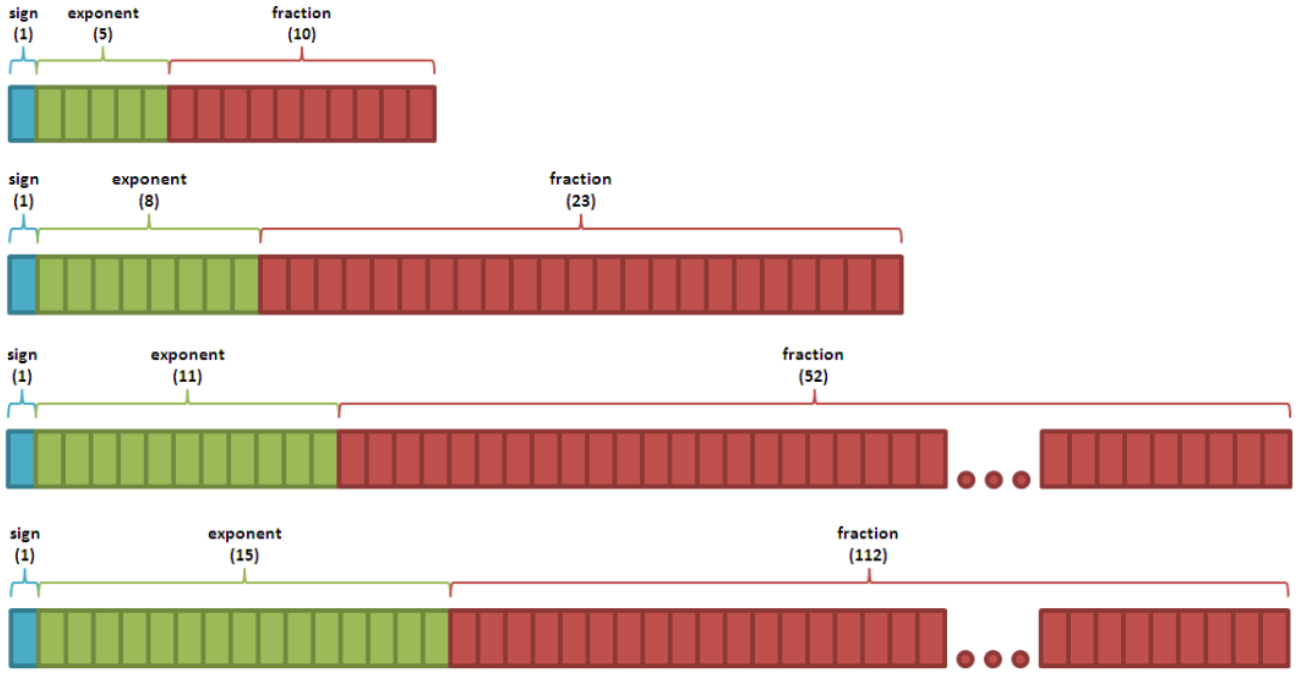
stack



SDU

# FPU – Floating Point Unit

- Floating-point is a way to represent real numbers on computers
- IEEE floating-point formats:
  - Half (16-bit)
  - Single (32-bit)
  - Double (64-bit)
  - Quadruple (128-bit)

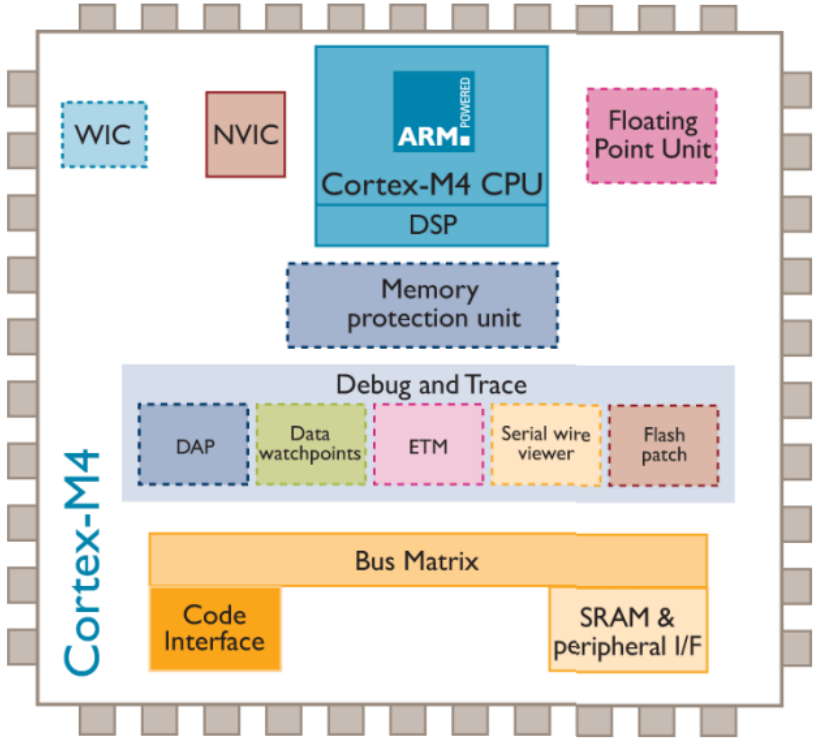





SDU

# FPU – Floating Point Unit

- The FPU provides floating-point computation functionality that is compliant with the IEEE 754 standard
- Enables conversions between fixed-point and floating-point data formats, and floating-point constant instructions
- The Cortex-M4F FPU fully supports single precision:
  - Add
  - Subtract
  - Multiply
  - Divide
  - Single cycle multiply and accumulate (MAC)
  - Square root



# FPU Usage

- The FPU is disabled from reset. You must enable it* before you can use any floating-point instructions.



SDU

# FPU Usage

- Find exercise here on page 9-8

```c
#include <stdint.h>
#include <stdbool.h>
#include <math.h>
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/fpu.h"
#include "driverlib/sysctl.h"
#include "driverlib/rom.h"

#ifndef M_PI
#define M_PI                    3.14159265358979323846
#endif

#define SERIES_LENGTH 100
float gSeriesData[SERIES_LENGTH];

int32_t i32DataCount = 0;

int main(void)
{
    float fRadians;

    ROM_FPULazyStackingEnable();
    ROM_FPUEnable();

    ROM_SysCtlClockSet(SYSCTL_SYSDIV_4 | SYSCTL_USE_PLL | SYSCTL_XTAL_16MHZ | SYSCTL_OSC_MAIN);

    fRadians = ((2 * M_PI) / SERIES_LENGTH);

    while(i32DataCount < SERIES_LENGTH)
    {
        gSeriesData[i32DataCount] = sinf(fRadians * i32DataCount);
        i32DataCount++;
    }

    while(1)
    {
    }
}
```



SDU

# The end!



SDU